

KURA.

Sistema de Gestão Veterinária Digital

FIAP Challenge 2026 · Parceiro: Clyvo Vet

DevOps Tools & Cloud Computing · Turma 2TDS · 2026

.NET 10

Spring Boot 3.2

FastAPI

Oracle XE

Docker

Azure

NOME	RM	RESPONSABILIDADE
Felipe Ferrete Soares Lemes	RM562999	Líder Técnico · .NET · IoT · IA
Clayton Alves	RM562285	DevOps · BD
Nikolas Brisola	RM564371	Java · Backend Tutor
Guilherme Sola	RM563674	Mobile Tutor · UX
Gustavo Bosak	RM566315	Mobile Clínica · QA

01

Descrição do Projeto

O **KURA** é um sistema de gestão veterinária digital desenvolvido para a Clyvo Vet, com foco em continuidade do cuidado e engajamento na jornada de saúde do pet. A plataforma resolve um problema central da veterinária moderna: a jornada do pet é episódica e reativa — o tutor só interage com a clínica em momentos de crise.

O KURA transforma isso em uma experiência contínua, preventiva e inteligente.

Microsserviços do Sistema

Serviço	Tecnologia	Porta	Descrição
kura-api	ASP.NET Core 10 · EF Core · Clean Architecture	8080	Backend clínico: vets, pets, eventos, IoT, dashboard
kura-tutor	Spring Boot 3.2 · Java 21 · Flyway · Caffeine	8081	Portal tutor: auth JWT, agendamentos, timeline, LGPD
luna-ai	FastAPI · Python 3.12 · YOLOv8n · MobileNetV3	8000	Triagem IA via WhatsApp (Twilio) + ID de raça por foto
oracle-db	Oracle XE 21c slim	1521 / 9092	Banco relacional compartilhado · volume persistente

Ordem de Inicialização dos Containers

Ordem	Container	Condição / Healthcheck	Tempo Estimado
1	oracle-db	lsnrctl status xepdb1 (30s retries 30)	~5 min (1ª inicialização)
2	kura-api	depends_on oracle-db (healthy)	~60 s
3	kura-tutor	depends_on oracle-db (healthy)	~90 s
4	luna-ai	depends_on kura-api (healthy) + oracle-db (healthy)	~60 s

02

Benefícios para o Negócio

Para o Tutor (responsável pelo pet)

- Lembretes automáticos de vacinas via WhatsApp antes do vencimento, eliminando esquecimentos e melhorando a adesão ao calendário vacinal.
- Portal digital unificado para acompanhar timeline de saúde, agendamentos, exames e histórico clínico do pet.
- Identificação de raça por foto com recomendações preventivas personalizadas geradas pela IA Luna (YOLOv8n + MobileNetV3).
- Triagem inteligente: ao enviar mensagem pelo WhatsApp, recebe classificação automática de urgência antes de falar com o veterinário.

Para a Clínica Veterinária

- Dashboard operacional com alertas de temperatura em tempo real (IoT ESP32 via API Key), agenda do dia e métricas de atendimento.
- Redução de no-shows: lembretes automáticos de consultas e confirmação via WhatsApp sem necessidade de ligação manual.
- Maior recorrência e fidelização: contato proativo entre consultas aumenta o LTV e a satisfação do tutor.
- Gestão de consentimentos LGPD integrada - relatório de dados disponível para tutores (art. 18 LGPD).

Para a Clyvo Vet (plataforma B2B, SaaS)

- Diferencial competitivo: stack de IA própria (visão computacional + NLP) integrada ao sistema de gestão clínica.
- Dados longitudinais de saúde por raça viabilizam analytics preditivos e desenvolvimento de novos produtos.
- Modelo de receita recorrente: SaaS mensal por clínica com diferenciação por funcionalidades de IA.
- Compliance nativo: LGPD implementada em todos os serviços — diferencial para clínicas preocupadas com regulação.

03

Arquitetura do Sistema

O ecossistema do KURA é orquestrado em uma infraestrutura Cloud utilizando a Azure, com microsserviços containerizados via Docker. Abaixo, a topologia da rede e a comunicação entre os nós.

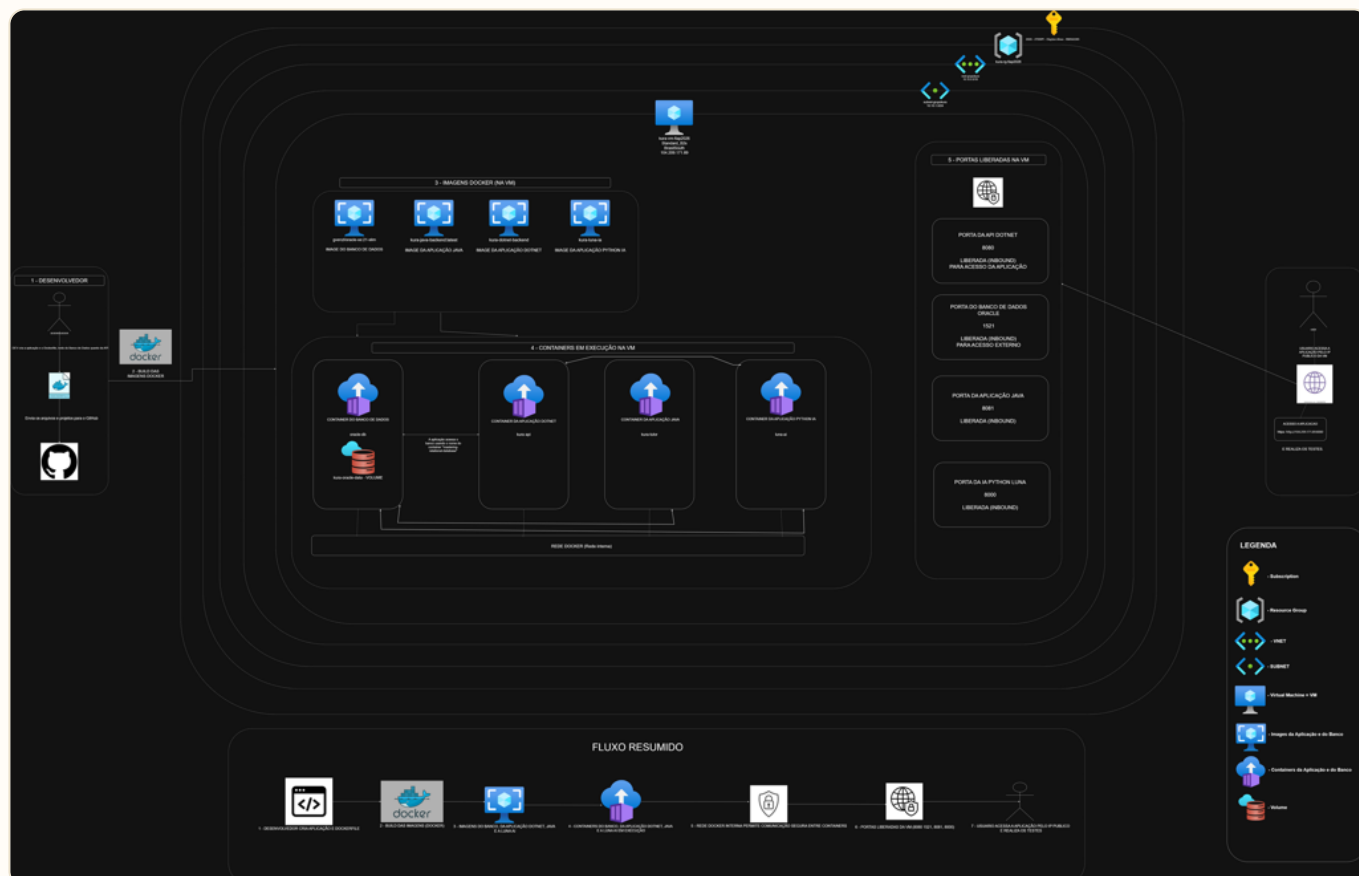


Figura 1: Diagrama de topologia dos containers na Máquina Virtual (Azure)

Fluxos Externos de Entrada

- **Veterinário / Gestor:** :8080 → kura-api (.NET) → /swagger
- **Tutor (App Mobile):** :8081 → kura-tutor (Java) → /api/swagger-ui/index.html
- **Dispositivo IoT ESP32:** :8080/api/v1/iot/leituras [Auth: X-API-Key]
- **Twilio WhatsApp:** :8000/webhook/twilio/whatsapp [Twilio Signature]

Comunicação Interna entre Containers (Rede kura_network)

Origem	Destino	Protocolo / Rota	Autenticação
luna-ai	kura-api	http://kura-api:8080	X-API-Key
kura-api	oracle-db	oracle-db:1521/XEPDB1	ODP.NET / APP_USER
kura-tutor	oracle-db	jdbc:oracle:thin:@//oracle-db:1521/XEPDB1	DB_USERNAME/PASSWORD
luna-ai	oracle-db	oracle-db:1521/XEPDB1	python-oracledb

Infraestrutura Docker

A arquitetura se baseia em uma Azure VM Ubuntu 22.04 LTS (Standard_D2s_v3) na região centralus. O docker-compose provisiona uma rede bridge isolada (kura_network).

Segurança: Usuários Não-Root (Rubrica 2.2)

Serviço	Implementação	Status
kura-api (.NET)	<code>RUN groupadd -r kura && useradd -r -g kura kura</code>	não-root
kura-tutor (Java)	<code>RUN groupadd --gid 1000 spring && useradd --uid 1000 spring</code>	não-root
luna-ai (Python)	<code>user: "1000:1000" definido no compose</code>	não-root
kura-app (Host)	<code>useradd --shell /bin/bash --create-home kura-app</code>	não-root

Volume Nomeado e Persistência (Rubrica 2.3)

O volume gerenciado `kura_oracle_data` mapeia os dados do banco para `/opt/oracle/oradata`. Isso garante que os dados da clínica persistam independentemente do ciclo de vida do container `oracle-db`.

05

Scripts Azure CLI

A automação foi desenvolvida no arquivo `script-azure.sh`, que provisiona a infraestrutura completa em 8 etapas sequenciais.

Passo	Ação	Recurso Azure
1	Criação do Resource Group kura-rg-fiap2026	Resource Groups
2	Provisionamento da VM Ubuntu 22.04 (Standard_D2s_v3)	Virtual Machines
3	Abertura de portas NSG (22, 8080, 8081, 8000, 9092)	Network Security Groups
4	Instalação remota do Docker Engine, Git e utilitários	VM Run Command
5	Criação do usuário não-root (kura-app) e grupos	User Management
6	Clonagem dos repositórios via submodules	Git clone
7	Build e inicialização dos containers (docker compose up -d)	Docker Compose
8	Validação e health checks HTTP contínuos	Health probes

Inovação Técnica: Helper `run_remote()`

Devido à limitação do comando `az vm run-command invoke`, que frequentemente retorna `exit 0` mesmo quando ocorre falha interna no script bash remoto, implementamos um padrão "sentinel".

```
run_remote() {
    local DESCRICAO="$1"
    local SCRIPT="$2"
    OUTPUT=$(az vm run-command invoke --scripts "$SCRIPT" --output json)

    if ! echo "$OUTPUT" | grep -q "STEP_OK"; then
        echo "FALHA REAL em: $DESCRICAO - interrompendo."
        exit 1
    fi
    echo "OK: $DESCRICAO"
}
```

Rotas Principais das APIs

.NET API (Backend Clínica) - Porta 8080

Método	Endpoint	Auth	Descrição
POST	/api/v1/auth/login	Pública	Login da clínica; retorna JWT
POST	/api/v1/tutores	JWT	Cria um tutor e gera UUID de convite
POST	/api/v1/agendamentos	JWT	Cria um agendamento
POST	/api/v1/iot/leituras	API Key	Ingestão de temperatura (IoT ESP32)

Java API (Backend Tutor) - Porta 8081

Método	Endpoint	Auth	Descrição
POST	/api/auth/register-invite	Pública	Onboarding do tutor com token UUID
GET	/api/pets/{id}/timeline	JWT	Linha do tempo de saúde do pet
GET	/api/tutores/{id}/lgpd/relatorio	JWT	Relatório LGPD completo do tutor (Art. 18)

07

Links dos Repositórios

- **Infraestrutura & DevOps:** github.com/KURA-Clyvo/DevOps-Cloud
- **Backend .NET (Clínica):** github.com/KURA-Clyvo/backend-clinica-dotnet
- **Backend Java (Tutor):** github.com/KURA-Clyvo/backend-tutor-java
- **Luna IA (Python):** github.com/KURA-Clyvo/kura-luna-ai

08

Evidência de Remoção da VM (Rubrica 4)

Para evitar cobranças residuais após a execução e validação do ambiente, os recursos instanciados na Azure foram devidamente deletados. O comando utilizado para a exclusão do grupo de recursos foi:

```
az group delete --name kura-rg-fiap2026 --yes --no-wait
```

Abaixo, a evidência extraída diretamente do portal Azure comprovando a ausência e remoção efetiva do Resource Group kura-rg-fiap2026.

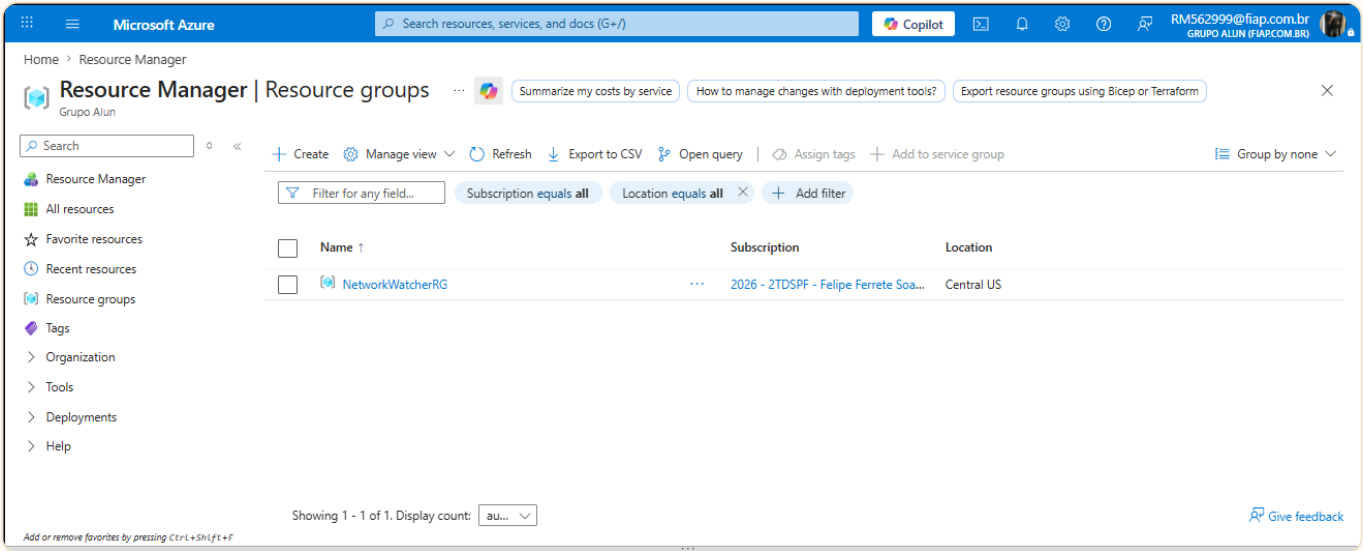


Figura 2: Portal Azure - Validação de remoção do Resource Group